

GenAI #1: Grok

Accessed through: Grok website

GenAI #2: Microsoft Copilot

Accessed through: Microsoft Copilot website

Prompt Used

The program should use classes/objects, not just a bunch of separate functions.

It needs to read puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, and puzzle5.txt. Each file has a 9x9 Sudoku puzzle. Numbers are already filled in and "_" means the spot is empty.

Use recursive depth-first search with backtracking to solve the puzzles.

For each puzzle, the program should:

- print the puzzle file name
- print the original puzzle
- find all possible solutions
- print every solution
- print "No solution found" if there is no solution
- print how many solutions were found

Put everything in one C++ file.

Keep the code basic and student-level, not overly advanced.

Add comments that explain the code clearly.

Copilot Generated Code

```
#include <iostream>
#include <fstream>
#include <vector>
```

```

#include <string>
using namespace std;
// Sudoku class handles reading, solving, and printing puzzles
class Sudoku {
private:
    vector<vector<char>>> board;
    vector<vector<vector<char>>>> solutions;
    // Check if placing num at (row, col) is valid
    bool isValid(int row, int col, char num) {
        // Check row
        for (int c = 0; c < 9; c++) {
            if (board[row][c] == num) {
                return false;
            }
        }
        // Check column
        for (int r = 0; r < 9; r++) {
            if (board[r][col] == num) {
                return false;
            }
        }
        // Check 3x3 box
        int startRow = (row / 3) * 3;
        int startCol = (col / 3) * 3;
        for (int r = startRow; r < startRow + 3; r++) {
            for (int c = startCol; c < startCol + 3; c++) {
                if (board[r][c] == num) {
                    return false;
                }
            }
        }
        return true;
    }
    // Recursive DFS with backtracking
    void solveHelper() {
        int row = -1;
        int col = -1;
        bool emptyFound = false;
        // Find first empty cell
        for (int r = 0; r < 9 && !emptyFound; r++) {
            for (int c = 0; c < 9; c++) {
                if (board[r][c] == '_') {
                    row = r;
                    col = c;
                    emptyFound = true;
                    break;
                }
            }
        }
    }
};

```

```

    }
}
}
// If no empty cells remain, a solution is found
if (!emptyFound) {
    solutions.push_back(board);
    return;
}
// Try numbers 1-9
for (char num = '1'; num <= '9'; num++) {
    if (isValid(row, col, num)) {
        board[row][col] = num;
        // Recur
        solveHelper();
        // Backtrack
        board[row][col] = '_';
    }
}
}
}
public:
    Sudoku() {
        board.resize(9, vector<char>(9));
    }
    // Read puzzle from file
    bool loadPuzzle(string filename) {
        ifstream file(filename);
        if (!file) {
            return false;
        }
        for (int r = 0; r < 9; r++) {
            for (int c = 0; c < 9; c++) {
                file >> board[r][c];
            }
        }
        file.close();
        return true;
    }
    // Print current puzzle board
    void printBoard() {
        for (int r = 0; r < 9; r++) {
            for (int c = 0; c < 9; c++) {
                cout << board[r][c] << " ";
            }
            cout << endl;
        }
    }
}

```

```

// Find all solutions
void solve() {
    solutions.clear();
    solveHelper();
}

// Print every solution found
void printSolutions() {
    if (solutions.empty()) {
        cout << "No solution found" << endl;
        return;
    }
    for (size_t i = 0; i < solutions.size(); i++) {
        cout << "\nSolution " << i + 1 << ":" << endl;
        for (int r = 0; r < 9; r++) {
            for (int c = 0; c < 9; c++) {
                cout << solutions[i][r][c] << " ";
            }
            cout << endl;
        }
    }
}

// Return number of solutions found
int getSolutionCount() {
    return solutions.size();
}

};

int main() {
    string files[5] = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    for (int i = 0; i < 5; i++) {
        Sudoku puzzle;
        cout << "===== " << endl;
        cout << "Puzzle File: " << files[i] << endl;
        cout << "===== " << endl;
        if (!puzzle.loadPuzzle(files[i])) {
            cout << "Could not open file.\n" << endl;
            continue;
        }
        cout << "\nOriginal Puzzle:\n" << endl;
        puzzle.printBoard();
        cout << "\nSearching for solutions...\n" << endl;
    }
}

```

```

    puzzle.solve();
    puzzle.printSolutions();
    cout << "\nTotal Solutions Found: "
          << puzzle.getSolutionCount() << endl;
    cout << endl;
}
return 0;
}

```

Grok Generated Code

```
// Sudoku Solver
```

```
// EECS 348 Assignment
```

```
// Uses a class and recursive depth-first search with backtracking
```

```
// to solve Sudoku puzzles from files.
```

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```

```
#include <vector>
```

```
using namespace std;
```

```
// =====
```

```
// Class that represents one Sudoku board
```

```
// =====
```

```
class Sudoku {
```

```
private:
```

```
    int board[9][9];    // the 9x9 grid (0 means empty)
```

```
    int solutionCount;  // how many solutions we have found
```

```
vector<string> solutions; // store each solution as a string so we can print them later
```

```
// Check whether it is safe to place 'num' at board[row][col]
```

```
bool isSafe(int row, int col, int num) {
```

```
    // Check the row
```

```
    for (int x = 0; x < 9; x++) {
```

```
        if (board[row][x] == num) {
```

```
            return false;
```

```
        }
```

```
    }
```

```
    // Check the column
```

```
    for (int x = 0; x < 9; x++) {
```

```
        if (board[x][col] == num) {
```

```
            return false;
```

```
        }
```

```
    }
```

```
    // Check the 3x3 box
```

```
    int startRow = row - row % 3;
```

```
    int startCol = col - col % 3;
```

```
    for (int i = 0; i < 3; i++) {
```

```
        for (int j = 0; j < 3; j++) {
```

```
            if (board[i + startRow][j + startCol] == num) {
```

```
                return false;
```

```
            }
```

```
}
```

```
}
```

```
return true; // safe to place the number
```

```
}
```

```
// Find the next empty cell (returns true if found)
```

```
// Sets row and col to the location of the empty cell
```

```
bool findEmpty(int &row, int &col) {
```

```
    for (row = 0; row < 9; row++) {
```

```
        for (col = 0; col < 9; col++) {
```

```
            if (board[row][col] == 0) {
```

```
                return true;
```

```
            }
```

```
        }
```

```
    }
```

```
return false; // no empty cell left --> board is full
```

```
}
```

```
// Recursive depth-first search with backtracking
```

```
// Finds ALL solutions
```

```
void solveRecursive() {
```

```
    int row, col;
```

```
// If there is no empty cell, we have found a complete solution
```

```
if (!findEmpty(row, col)) {
```

```

        solutionCount++;

        // Save the current board as a string so we can print it later
        string sol = "";
        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++) {
                sol += to_string(board[i][j]);

                if (j < 8) sol += " ";
            }

            sol += "\n";
        }

        solutions.push_back(sol);

        return;
    }
}

```

```

        // Try every number from 1 to 9 in this empty cell
        for (int num = 1; num <= 9; num++) {
            if (isSafe(row, col, num)) {
                board[row][col] = num;    // place the number

                solveRecursive();    // recurse

                board[row][col] = 0;    // backtrack (undo the move)
            }
        }
    }
}

```

```

public:

    // Constructor - initializes the board to all zeros

```

```

Sudoku() {
    for (int i = 0; i < 9; i++) {
        for (int j = 0; j < 9; j++) {
            board[i][j] = 0;
        }
    }
    solutionCount = 0;
}

```

```

// Load a puzzle from a file
// '_' means empty, digits 1-9 are filled cells
bool loadFromFile(const string &filename) {
    ifstream file(filename);
    if (!file.is_open()) {
        cout << "Error: Could not open file " << filename << endl;
        return false;
    }
}

```

```

string line;
int row = 0;
while (getline(file, line) && row < 9) {
    // Remove any extra spaces or carriage returns
    string clean = "";
    for (char c : line) {
        if (c != ' ' && c != '\r') {
            clean += c;
        }
    }
}

```

```
    }
```

```
    }
```

```
        // Make sure the line has at least 9 characters
```

```
        if (clean.length() < 9) {
```

```
            cout << "Error: Bad line in file " << filename << endl;
```

```
            return false;
```

```
        }
```

```
        for (int col = 0; col < 9; col++) {
```

```
            char ch = clean[col];
```

```
            if (ch == '_') {
```

```
                board[row][col] = 0;    // empty cell
```

```
            } else if (ch >= '1' && ch <= '9') {
```

```
                board[row][col] = ch - '0'; // convert char to int
```

```
            } else {
```

```
                cout << "Error: Invalid character in file " << filename << endl;
```

```
                return false;
```

```
            }
```

```
        }
```

```
        row++;
```

```
    }
```

```
    file.close();
```

```
    if (row != 9) {
```

```
    cout << "Error: File " << filename << " does not contain 9 rows." << endl;
```

```
    return false;
```

```
}
```

```
    return true;
```

```
}
```

```
// Print the current board in a nice format
```

```
void printBoard() const {
```

```
    for (int i = 0; i < 9; i++) {
```

```
        if (i % 3 == 0 && i != 0) {
```

```
            cout << "-----+-----+-----" << endl;
```

```
        }
```

```
        for (int j = 0; j < 9; j++) {
```

```
            if (j % 3 == 0 && j != 0) {
```

```
                cout << "| ";
```

```
            }
```

```
            if (board[i][j] == 0) {
```

```
                cout << "_ ";
```

```
            } else {
```

```
                cout << board[i][j] << " ";
```

```
            }
```

```
        }
```

```
    cout << endl;
```

```
}
```

```
}
```

```
// Solve the puzzle and collect all solutions
```

```
void solve() {
```

```
    solutionCount = 0;
```

```
    solutions.clear();
```

```
    solveRecursive();
```

```
}
```

```
// Print all solutions that were found
```

```
void printSolutions() const {
```

```
    if (solutionCount == 0) {
```

```
        cout << "No solution found" << endl;
```

```
    } else {
```

```
        for (int i = 0; i < solutionCount; i++) {
```

```
            cout << "Solution " << (i + 1) << ":" << endl;
```

```
            cout << solutions[i];
```

```
            cout << endl;
```

```
        }
```

```
    }
```

```
    cout << "Number of solutions found: " << solutionCount << endl;
```

```
}
```

```
};
```

```
// =====
```

```
// Main program
```

```
// =====
```

```
int main() {  
    // List of puzzle files to process  
    string files[] = {  
        "puzzle1.txt",  
        "puzzle2.txt",  
        "puzzle3.txt",  
        "puzzle4.txt",  
        "puzzle5.txt"  
    };  
  
    // Process each puzzle file  
    for (int f = 0; f < 5; f++) {  
        string filename = files[f];  
  
        cout << "======" << endl;  
        cout << "Puzzle file: " << filename << endl;  
        cout << "======" << endl;  
  
        Sudoku puzzle;  
  
        // Try to load the puzzle  
        if (!puzzle.loadFromFile(filename)) {  
            cout << "Skipping this puzzle because of a file error." << endl << endl;  
            continue;  
        }  
    }
```

```

// Print the original puzzle

cout << "Original puzzle:" << endl;

puzzle.printBoard();

cout << endl;


// Solve it (finds all solutions)

puzzle.solve();


// Print every solution (or "No solution found")

puzzle.printSolutions();

cout << endl;

}


return 0;

}

```

Code Comparison

Correctness

Both programs use a Sudoku class and recursive backtracking. They check the row, column, and 3x3 block before placing a number. They also keep searching after finding a solution, so they can find multiple solutions.

Both programs worked with the provided puzzles and followed the main assignment requirements.

Execution Time

Both programs use basically the same brute-force backtracking method, so their execution times should be similar. The actual time depends on how many empty cells and possible choices each puzzle has.

Neither program uses a major optimization like always choosing the empty cell with the fewest possible numbers.

Space Complexity

Both programs use extra memory because they store all the solutions they find. They also use memory for the recursive calls while solving the puzzle.

Copilot stores solutions as 2D character boards. Grok converts each solution into a string before storing it.

Maintainability

I think Copilot is easier to maintain. The puzzle files already use characters, including _ for empty spaces, and Copilot keeps the board as characters. Grok changes _ to 0 and later converts the solved board back into a string.

Copilot's approach is more direct and has less conversion between different formats.

Code Selected

I chose Copilot as the base for my final program.

Both programs use almost the same backtracking idea, so there was not a big difference in correctness or execution time. I mainly chose Copilot because the code is simpler for me to understand and change. It also reads the puzzle characters directly instead of converting the blanks to 0 and later converting the solution back into a string.

Improvements

I made changes to the Copilot code before using it as my final program.

Correctness: I added better checks when reading the puzzle files so invalid input can be detected instead of being treated like a normal puzzle.

Execution Time: I improved how the program chooses an empty cell by looking for a cell with fewer possible values. This can reduce the amount of unnecessary backtracking.

Space Complexity: Instead of keeping every completed solution in memory, I changed the program so solutions can be printed when they are found while only keeping track of the number of solutions.

Maintainability: I added the required prologue, improved the comments in my own words, and made the names and organization of the code easier for me to understand.